

Problem Set 2: Visualization

Readings: Newman Ch. 3

Due: Wednesday, September 16th 11:59 PM

Instructions: Work through each problem, following the instructions for the requirements of the code and their outputs. Some starting code is provided for each problem on the course website. Problem sets should be submitted in one batch, with each problem uploaded as it's own python script (.py file) on the HuskyCT assignment page with the filename: `ps[PROBLEM SET NUMBER]_[QUESTION NUMBER]_[LASTNAME].py`.

Parameters:

- (1) Submitted solutions are restricted to imports of external functions from standard built-in Python libraries (i.e. `math`, `time`, `os`, `sys`, etc.). In addition, imports from `matplotlib.pyplot` and `numpy` are permitted.
 - (2) All plots should have both axes clearly labeled and include a title. Plots should include either legends or annotations for differentiating plot elements, when applicable.
-

1. PLOTTING EXPERIMENTAL DATA

In the online problem-set resources, you will find a file which contains the observed number of sunspots on the Sun for each month since January 1749. The file contains two columns of numbers, the first being the month and the second being the sunspot number.

Write a program to read in the data and make a plot of the number of sunspots as a function of time. The program should:

- a. Read in the data from `sunspots.txt`
- b. Include a function to calculate the running average Y_k of the data y_k defined by:

$$Y_k = \frac{1}{2r} \sum_{m=-r}^r y_{k+m},$$

for $r = 5$ in this case.

- c. Plot both the original data and the running average on the same graph for only the range covered by the first 1000 data points.
- d. Save the figure as `sunspots_[LASTNAME].png`

2. SCANNING TUNNELING MICROSCOPE

A scanning tunneling microscope (STM) is a device that measures the shape of a surface at the atomic level by tracking a sharp tip over the surface and measuring quantum tunneling current as a function of position. In the online problem-set resources, you will find a file which contains a grid of values from scanning tunneling microscope measurements of the (111) surface of silicon, where the values represent the height of the surface at each position in the grid. Write a program that reads the data contained in the file and makes a density plot of the values. The program should:

- a. Read the data from `stm.txt` and print the dimensions of the grid.

- b. Plot a 2D density plot of the values, in which the height of the surface at a certain position in the grid is represented by a color. The silicon surface should be shown clearly. (Explore various options from the matplotlib library).
- c. Save the figure as `stm_[LASTNAME].png`.

3. THE MANDELBROT SET

The Mandelbrot set, named after its discoverer, the French mathematician Benoît Mandelbrot, is a *fractal*, an infinitely ramified mathematical object that contains structure within structure within structure, as deep as we care to look. The definition of the Mandelbrot set is in terms of complex numbers as follows.

Consider the equation

$$z' = z^2 + c,$$

where z is a complex number and c is a complex constant.

For any given value of c this equation turns an input number z into an output number z' . The definition of the Mandelbrot set involves the repeated iteration of this equation: we take an initial starting value of z and feed it into the equation to get a new value z' . Then we take that value and feed it in again to get another value, and so forth. The Mandelbrot set is the set of points in the complex plane that satisfies the following definition:

For a given complex value of c , start with $z = 0$ and iterate repeatedly. If the magnitude $|z|$ of the resulting value is ever greater than 2, then the point in the complex plane at position c is not in the Mandelbrot set, otherwise it is in the set.

In order to use this definition one would, in principle, have to iterate infinitely many times to prove that a point is in the Mandelbrot set, since a point is in the set only if the iteration never passes $|z| = 2$ ever. In practice, however, one usually just performs some large number of iterations, say 100, and if $|z|$ hasn't exceeded 2 by that point then we call that good enough.

Write a program to make an image of bounds of the Mandelbrot set. The program should:

- a. Define a function `is_mandelbrot()` to determine whether a point is inside or outside the set. Based on the result of your function, print a determination (True or False), for $x = -1.25, y = 0.0; c = -1.25 + 0j$.
- b. Perform the iteration for all values of $c = x + iy$ on an $N \times N$ grid spanning the region where $-2 \leq x \leq 2$ and $-2 \leq y \leq 2$.
- c. Make a density plot in which grid points inside the Mandelbrot set are colored black and those outside are colored white.
- d. Save the figure as `mandelbrot_[LASTNAME].png`

Hint: You will probably find it useful to start off with quite a coarse grid, i.e., with a small value of N —perhaps $N = 100$ —so that your program runs quickly while you are testing it. Once you are sure it is working correctly, increase the value of N to produce a final high-quality image of the shape of the set.

bonus: Rather than just coloring by whether a space in the grid is outside of or inside the set, color the points according to the number of iterations of the equation done before $|z|$ becomes greater than 2 (or the maximum number of iterations if $|z|$ never becomes greater than 2). You may wish to use a log scale here.

4. LEAST-SQUARES FITTING AND THE PHOTOELECTRIC EFFECT

It's a common situation in physics that an experiment produces data that lies roughly on a straight line, like the dots in this figure:

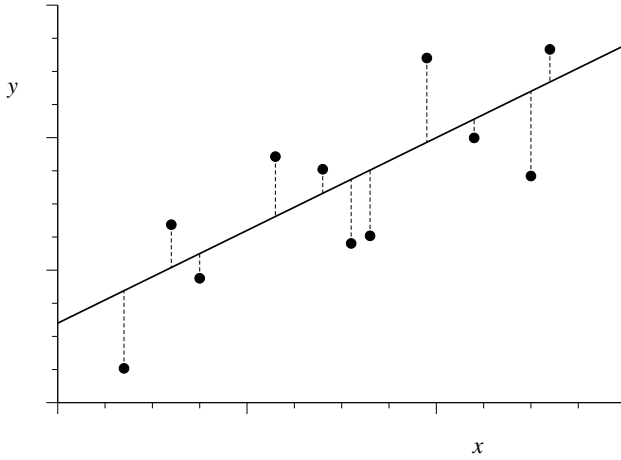


Figure 1: The solid line here represents the underlying model, which we usually don't know, and the points representing the measured data lie roughly along the line but don't fall exactly on it, typically because of measurement error.

The straight line can be represented in the familiar form $y = mx + c$ and a frequent question is what the appropriate values of the slope m and intercept c are that correspond to the measured data. Since the data don't fall perfectly on a straight line, there is no perfect answer to such a question, but we can find the straight line that gives the best compromise fit to the data. The standard technique for doing this is the **method of least squares**.

Suppose we make some guess about the parameters m and c for the model straight line. We then calculate the vertical distances between the data points and that line, as represented by the short vertical lines in the figure, then we calculate the sum of the squares of those distances, which we denote χ^2 . If we have N data points with coordinates (x_i, y_i) , then the total χ^2 is given by:

$$\chi^2 = \sum_{i=1}^N (mx_i + c - y_i)^2.$$

The least-squares fit of the straight line to the data is the straight line that minimizes this total squared distance from data to model. We find the minimum by differentiating with respect to both m and c and setting the derivatives to zero, which gives

$$\begin{aligned} m \sum_{i=1}^N x_i^2 + c \sum_{i=1}^N x_i - \sum_{i=1}^N x_i y_i &= 0, \\ m \sum_{i=1}^N x_i + cN - \sum_{i=1}^N y_i &= 0. \end{aligned}$$

For convenience, let us define the following quantities:

$$E_x = \frac{1}{N} \sum_{i=1}^N x_i, \quad E_y = \frac{1}{N} \sum_{i=1}^N y_i, \quad E_{xx} = \frac{1}{N} \sum_{i=1}^N x_i^2, \quad E_{xy} = \frac{1}{N} \sum_{i=1}^N x_i y_i,$$

in terms of which our equations can be written

$$\begin{aligned} mE_{xx} + cE_x &= E_{xy}, \\ mE_x + c &= E_y. \end{aligned}$$

Solving these equations simultaneously for m and c now gives

$$m = \frac{E_{xy} - E_x E_y}{E_{xx} - E_x^2}, \quad c = \frac{E_{xx} E_y - E_x E_{xy}}{E_{xx} - E_x^2}.$$

These are the equations for the least-squares fit of a straight line to N data points. They tell you the values of m and c for the line that best fits the given data.

The photoelectric effect can be described by the equation, first given by Albert Einstein in 1905, as:

$$V = \frac{h}{e} \nu - \phi$$

Where an incoming photon with sufficiently high energy $h\nu$ can liberate an electron from a metal surface with work function ϕ . Thus, the energy of the liberated electron is measured by the voltage V , where e is the charge of the electron. Write a program to plot the data from the Millikan experiment, included in the online resources, and fit the value of the Planck constant h by the method of least-squares. The program should:

- Read in the data from `millikan.txt`. The file contains two columns of numbers, giving the x and y coordinates of a set of data points. The first column represents frequencies ν in hertz and second column the measured voltages V in volts from the historic experiment by Robert Millikan.
- Define a function `fit_lsq(x,y)` to calculate the quantities E_x , E_y , E_{xx} , and E_{xy} defined above, and from them calculate and print out the slope m and intercept c of the best-fit line.
- Plot on the same graph the data as individual points and the line of best fit corresponding to your fit above.
- Calculate and print out the Planck constant based on the measurements, given the charge of an electron $e = 1.602 \times 10^{-19}$ C. Print the percent error on the Planck constant compared to the accepted value. You should get a result within a couple of percent of the accepted value.
- Save your figure as `millikan_[LASTNAME].png`.

This calculation is essentially the same as the one that Millikan himself used to determine the value of Planck's constant, although, lacking a computer, he fitted his straight line to the data by eye. In part for this work, Millikan was awarded the Nobel prize in physics in 1923.